🖼 Image Optimization Pipeline

Interview Redeployment Guide — Every step explained with full context

AWS Lambda · S3 (3 Buckets) · API Gateway REST · Python 3.11 · Pillow / PIL · Pre-signed URLs · S3 Event Triggers

CONTENTS

1. How This Differs from LinkForge
2. Architecture Overview
3. Prerequisites & One-time Setup
4. Phase 1 — S3 Buckets (3 buckets)
5. Phase 2 — IAM Role for Lambda
6. Phase 3 — Lambda Functions (3 functions)
7. Phase 4 — S3 Event Trigger
8. Phase 5 — API Gateway (REST API)
9. Phase 6 — Frontend to S3
10. Verification & Testing
11. Data Flow & Key Configs Reference
12. Troubleshooting
13. Interview Master Checklist
14. Talking Points

1 How This Project Differs from LinkForge — Read First

WHY READ THIS SECTION?

If you recently deployed LinkForge, your muscle memory will trip you up here. This project uses a completely different architecture pattern. The table below shows where your assumptions will be wrong.

LinkForge Pro

CloudFormation creates everything in one command. HTTP API (v2). Lambda wired to API via integrations. Frontend points to API Gateway URL for all operations.

Image Optimization Pipeline

No CloudFormation — everything created manually with CLI. REST API (v1). Lambda triggered by S3 events,

not API Gateway. Frontend uploads DIRECTLY to S3 (bypasses API Gateway entirely for the actual file).

Aspect	LinkForge	Image Pipeline
IaC	CloudFormation (1 YAML file)	Manual CLI commands only — no template
API type	HTTP API v2 (apigatewayv2)	REST API v1 (apigateway) — different CLI commands
Lambda trigger	API Gateway route → Lambda	S3 ObjectCreated event → Lambda (automatic, no API route)
File upload path	POST to API → Lambda → DynamoDB	Frontend → API Gateway → Lambda generates pre-signed URL → Frontend uploads directly to S3
S3 buckets	1 bucket (frontend only)	3 buckets: input, output, frontend
Pillow / PIL	Not used	Core dependency — must be bundled in Lambda ZIP or added as a Layer
Region	us-east-1	ap-south-1 (Mumbai) — all commands must match
API CORS	Configured at API Gateway level in CloudFormation	Must be manually configured with OPTIONS methods on each resource

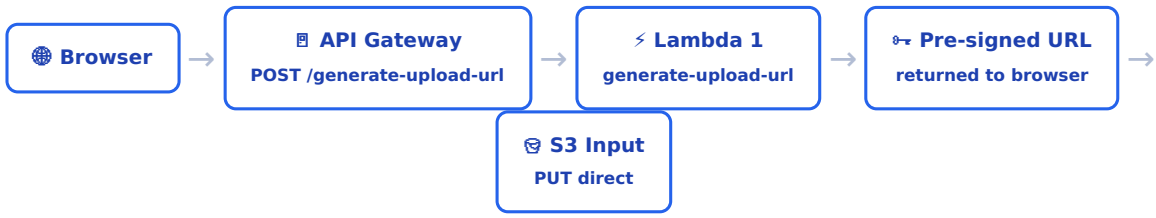
⚠ **Region is ap-south-1 throughout this project.**
Every CLI command needs `--region ap-south-1`. The Lambda functions, S3 buckets, and API Gateway are all in Mumbai. If you accidentally deploy to us-east-1, nothing will connect.

2 Architecture Overview — Two Separate Flows

WHY TWO FLOWS?

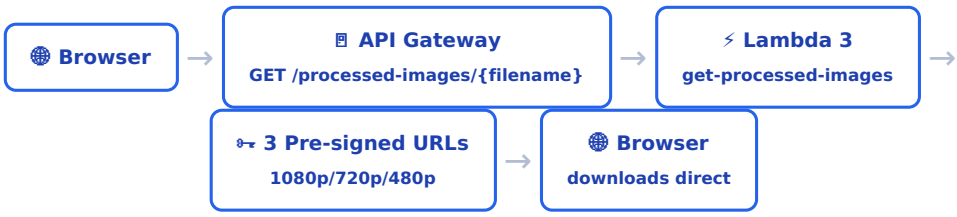
This project has an **upload flow** and a **download flow** — and the image processing is fully automated in between, triggered by S3 itself. Understanding that the API Gateway is NOT involved in the actual file transfer is the single most important architectural insight.

Upload Flow



Auto-Processing Flow (triggered automatically)

Download Flow



Resource	Name / ID	Purpose
S3 Input Bucket	image-pipeline-input-$SUFFIX	Receives original uploads via pre-signed PUT URL. Fires S3 event on every new object.
S3 Output Bucket	image-pipeline-output-$SUFFIX	Stores processed images in 3 subfolders: 1080p/, 720p/, 480p/
S3 Frontend Bucket	image-pipeline-frontend-$SUFFIX	Static website hosting for index.html, styles.css, app.js
Lambda 1	generate-upload-url	Generates a 15-minute pre-signed S3 PUT URL. Called by API Gateway.
Lambda 2	ImageProcessorLambda-Shivam	Downloads image, runs Pillow resize + compress, uploads 3 versions. Triggered by S3 event — never called by API.
Lambda 3	get-processed-images	Generates 3 pre-signed S3 GET URLs (1 hour). Called by API Gateway.
API Gateway	ImagePipelineAPI (REST v1)	Two endpoints only — URL generation and URL retrieval. Never touches actual image bytes.
IAM Role	Lambda-Image-Processing-Role	Grants all 3 Lambda functions permission to read/write S3 and write CloudWatch logs.

3 Prerequisites & Setup

1 Verify AWS CLI and credentials

```
aws --version          # Need 2.x
aws sts get-caller-identity   # Confirms credentials work — note the Account ID
```

> **WHY CHECK CREDENTIALS FIRST?**

All 6 deployment phases below require AWS CLI access. The Account ID returned here is used when constructing Lambda ARNs for API Gateway integration in Phase 5. Save it: `ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)`

2 Verify Python and pip

```
python3 --version     # Need 3.9+
pip3 --version
```

WHY PYTHON SPECIFICALLY?

Lambda 2 (ImageProcessor) uses Pillow (PIL) for image processing. When you create the Lambda deployment ZIP, you must install Pillow locally first and bundle it with the code — otherwise Lambda will throw ImportError. Pillow is NOT pre-installed in any Lambda runtime.

3 Set your region variable — use it everywhere

```
export REGION="ap-south-1"
export ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
echo "Region: $REGION | Account: $ACCOUNT_ID"
```

WHY SET THESE AS SHELL VARIABLES?

This project uses ap-south-1 (Mumbai) — NOT us-east-1 like LinkForge. Every aws CLI command in this guide uses `$REGION`. If you type the region wrong even once, resources land in different regions and cannot communicate. Setting it once as a variable prevents all typos. The `$ACCOUNT_ID` variable is used in Phase 5 when building Lambda function ARNs for API Gateway.

4 Clone repo and understand the structure

```
git clone https://github.com/yourusername/image-optimization-pipeline.git
cd image-optimization-pipeline
```

KEY FILES YOU WILL BE DEPLOYING

`lambda-functions/generate-upload-url/lambda_function.py` — Lambda 1 code (simple, no dependencies)

`lambda-functions/get-processed-images/lambda_function.py` — Lambda 3 code (simple, no

dependencies)

`lambda-functions/image-processor/lambda_function.py` — Lambda 2 code (requires Pillow bundled in ZIP)

`frontend/index.html, styles.css, app.js` — the static web app that goes to the frontend S3 bucket

4 Phase 1 — Create 3 S3 Buckets

Why S3 first? The IAM Role (Phase 2) references bucket ARNs in its permissions. Lambda functions (Phase 3) reference bucket names in their environment variables. The S3 event trigger (Phase 4) is attached to the input bucket. Everything downstream depends on the buckets existing first.

⚠ **Global rule for this entire guide — single quotes vs double quotes in zsh/bash:**

Many commands below pass JSON to the AWS CLI via `--policy`, `--policy-document`, `--cors-configuration`, etc. The quoting rule is:

- **No variables inside JSON → single quotes are fine:**
  `'{"key": "value"}'`
- **Variables inside JSON → must use double quotes + escape inner quotes:**
  `"{\"key\": \"${MY_VAR}\"}"`

Single quotes in zsh prevent ALL variable expansion — `'$BUCKET'` passes the literal string `$BUCKET` to AWS, not your bucket name. This causes `MalformedPolicy` errors on IAM and S3 policy commands, and `InvalidParameter` errors on notification configs. Every JSON block in this guide that contains a `$VARIABLE` reference uses double quotes.

1 Create the 3 buckets

⚠ **Set your suffix variable FIRST — you will use it in every command throughout this guide:**

```
# Replace "anshuman" with your own name/initials
export SUFFIX="anshuman"
export INPUT_BUCKET="image-pipeline-input-$SUFFIX"
export OUTPUT_BUCKET="image-pipeline-output-$SUFFIX"
export FRONTEND_BUCKET="image-pipeline-frontend-$SUFFIX"
echo "$INPUT_BUCKET | $OUTPUT_BUCKET | $FRONTEND_BUCKET"
```

Every subsequent command in this guide uses `$INPUT_BUCKET`, `$OUTPUT_BUCKET`, and

`$FRONTEND_BUCKET` — not the hardcoded `-sid` names. If you open a new terminal session, re-export these variables before continuing.

```
# Input bucket — receives original uploaded images
aws s3api create-bucket \
  --bucket $INPUT_BUCKET \
  --region ap-south-1 \
  --create-bucket-configuration LocationConstraint=ap-south-1

# Output bucket — stores processed images in 1080p/, 720p/, 480p/ folders
aws s3api create-bucket \
  --bucket $OUTPUT_BUCKET \
  --region ap-south-1 \
  --create-bucket-configuration LocationConstraint=ap-south-1

# Frontend bucket — hosts the static website
aws s3api create-bucket \
  --bucket $FRONTEND_BUCKET \
  --region ap-south-1 \
  --create-bucket-configuration LocationConstraint=ap-south-1
```

Flag	What it does and why it's needed
`create-bucket`	Creates an S3 bucket. Bucket names must be globally unique across ALL AWS accounts worldwide — not just your account. The original "-sid" names are already claimed by the original team member Siddhant. You must use your own unique suffix every time you redeploy. **Success response is `{"Location": "http://bucketname.s3.amazonaws.com/"}` — this is correct.**
`$INPUT_BUCKET / $OUTPUT_BUCKET / $FRONTEND_BUCKET`	Shell variables set above. Using variables instead of hardcoded names means every downstream command (Lambda env vars, IAM policy, S3 event trigger) automatically uses the same consistent name. One source of truth — change the SUFFIX once and everything cascades correctly.
`--create-bucket-configuration LocationConstraint=ap-south-1`	Required for all regions EXCEPT us-east-1. Without this flag in ap-south-1, the command fails with an "InvalidLocationConstraint" error. us-east-1 is the default region for S3 and does not need LocationConstraint. Every other region requires it explicitly.

Issue Hit: BucketAlreadyExists for all 3 buckets

```
An error occurred (BucketAlreadyExists) when calling the CreateBucket operation:
The requested bucket name is not available. The bucket namespace is shared by
```

```
all users of the system. Please select a different name and try again.
```

Why it happened:

S3 bucket names are a single global namespace shared across every AWS account on the planet — not just yours. The names `image-pipeline-input-sid`, `image-pipeline-output-sid`, and `image-pipeline-frontend-sid` were created by the original team member (Siddhant) and are permanently claimed in his account. No one else can ever create a bucket with those exact names.

Fix — use your own suffix:

```
export SUFFIX="anshuman"    # ← your name/initials
export INPUT_BUCKET="image-pipeline-input-$SUFFIX"
export OUTPUT_BUCKET="image-pipeline-output-$SUFFIX"
export FRONTEND_BUCKET="image-pipeline-frontend-$SUFFIX"

aws s3api create-bucket --bucket $INPUT_BUCKET \
  --region ap-south-1 --create-bucket-configuration LocationConstraint=ap-south-1

aws s3api create-bucket --bucket $OUTPUT_BUCKET \
  --region ap-south-1 --create-bucket-configuration LocationConstraint=ap-south-1

aws s3api create-bucket --bucket $FRONTEND_BUCKET \
  --region ap-south-1 --create-bucket-configuration LocationConstraint=ap-south-1
```

What changed downstream:

Because you changed the bucket names, you must use `$INPUT_BUCKET` / `$OUTPUT_BUCKET` / `$FRONTEND_BUCKET` in every subsequent step — the IAM policy Resource ARNs, Lambda environment variables, S3 event notification, CORS configs, and the frontend website URL. The shell variables handle this automatically as long as they are set in your terminal session.

Remember for interviews:

Always use a personal suffix when redeploying any project. Bucket names from a team project are already claimed. The safest pattern: `projectname-purpose-yourname`.

WHY 3 SEPARATE BUCKETS INSTEAD OF ONE?

Input bucket: Needs PUT permission for pre-signed uploads and fires S3 events. Mixing it with processed images would cause the event trigger to fire on its own outputs — an infinite processing loop.

Output bucket: Needs GET permission for pre-signed downloads. Organized with resolution subfolders (1080p/, 720p/, 480p/) so the get-processed-images Lambda can construct predictable keys.

Frontend bucket: Needs static website hosting enabled with public read. Mixing static site files with image data would be confusing and could accidentally expose images.

2 Enable static website hosting on the frontend bucket

```
aws s3 website s3://$FRONTEND_BUCKET/ \
  --index-document index.html \
  --error-document index.html \
  --region ap-south-1
```

> **WHAT DOES WEBSITE HOSTING DO?**
>
> Normally S3 objects are accessed via the REST API endpoint (e.g. s3.amazonaws.com/bucket/key). With website hosting enabled, S3 creates a separate website endpoint (e.g. bucket.s3-website.ap-south-1.amazonaws.com) that behaves like a web server — it serves index.html for the root URL, handles 404s with the error document, and serves files with correct content-type headers. This is required for a browser to load the app correctly.

3 Make the frontend bucket publicly readable

```
# Step 3a: Disable block public access (S3 blocks public access by default since 2023)
aws s3api put-public-access-block \
  --bucket $FRONTEND_BUCKET \
  --public-access-block-configuration \
  BlockPublicAcls=false,IgnorePublicAcls=false,BlockPublicPolicy=false,RestrictPublicBuckets=fa
  --region ap-south-1

# Step 3b: Attach a bucket policy allowing public GET
# IMPORTANT: Use double quotes (not single quotes) so ${FRONTEND_BUCKET} expands correctly
aws s3api put-bucket-policy \
  --bucket $FRONTEND_BUCKET \
  --region ap-south-1 \
  --policy "{
    \"Version\": \"2012-10-17\",
    \"Statement\": [{
      \"Sid\": \"PublicReadGetObject\",
      \"Effect\": \"Allow\",
      \"Principal\": \"*\",
      \"Action\": \"s3:GetObject\",
      \"Resource\": \"arn:aws:s3:::${FRONTEND_BUCKET}/*\"
    }]
  }"
```

Issue 1 Hit: zsh: unknown file attribute: 3

```
zsh: unknown file attribute: 3
```

Why it happened:

zsh interpreted the number in the `--public-access-block-configuration` flag value (BlockPublicAcls=false,...) differently when pasted with the backslash continuation on the same line. This error is harmless — it means zsh got confused about the line break, but the AWS command itself still executed successfully. No AWS error = Step 3a completed fine. You can verify with:

```
aws s3api get-public-access-block --bucket $FRONTEND_BUCKET --region ap-south-1
```

All four values should return `false`. The fix in the guide above puts the configuration value on its own indented line without the trailing backslash issue.

Issue 2 Hit: MalformedPolicy — Policy has invalid resource

```
An error occurred (MalformedPolicy) when calling the PutBucketPolicy operation:
Policy has invalid resource
```

Why it happened:

The bucket policy JSON was wrapped in **single quotes** (`'...'`). In zsh (and bash), single quotes prevent ALL variable expansion — the string `$FRONTEND_BUCKET` was passed literally to AWS instead of being expanded to `image-pipeline-frontend-anshuman`. AWS received the ARN as `arn:aws:s3:::$FRONTEND_BUCKET/*` which is not a valid ARN, so it rejected the policy.

The rule — single quotes vs double quotes in shell:

- `'$VAR'` → literal string `$VAR` (no expansion) — use for static JSON with no variables
- `"$VAR"` or `"${VAR}"` → expands to the variable's value — required whenever your JSON contains shell variables

Fix — use double quotes and escape inner quotes with backslashes:

```
aws s3api put-bucket-policy \
  --bucket $FRONTEND_BUCKET \
  --region ap-south-1 \
  --policy "{
    \"Version\": \"2012-10-17\",
    \"Statement\": [{
      \"Sid\": \"PublicReadGetObject\",
      \"Effect\": \"Allow\",
      \"Principal\": \"*\",
      \"Action\": \"s3:GetObject\",
      \"Resource\": \"arn:aws:s3:::${FRONTEND_BUCKET}/*\"
    }]
  }"
```

Success response:

No output at all — an empty response from `put-bucket-policy` means success. AWS only returns output on failure.

Remember for interviews:

This single-quote vs double-quote issue will appear in EVERY step that passes JSON with shell variables to the AWS CLI — the IAM policy (Phase 2), the S3 event notification (Phase 4), and the API Gateway CORS integration responses (Phase 5). Always use double quotes + escaped inner quotes when your JSON contains `$VARIABLE` references.

WHY TWO STEPS FOR PUBLIC ACCESS?

AWS introduced "Block Public Access" settings in 2018 as a safety guard — even if you add a bucket policy, the block settings override it and prevent public access. You must explicitly disable the block settings first, THEN add the bucket policy. Doing only one of these is a common mistake that results in 403 errors when trying to load the frontend website.

The bucket policy uses `Principal: "*"` (anyone) and `Action: s3:GetObject` (read-only) on all objects (`/*`). This allows any browser to fetch HTML/CSS/JS files — which is exactly what a public website needs.

How to verify it worked: `aws s3api get-bucket-policy --bucket $FRONTEND_BUCKET --region ap-south-1` — the returned policy should show your bucket name expanded correctly (not `$FRONTEND_BUCKET` literally).

4 Enable CORS on the input bucket (browsers need this for direct PUT)

```
# Use double quotes so $INPUT_BUCKET expands correctly
aws s3api put-bucket-cors \
  --bucket $INPUT_BUCKET \
  --region ap-south-1 \
  --cors-configuration '{
    "CORSRules": [{
      "AllowedOrigins": ["*"],
      "AllowedMethods": ["PUT", "POST", "GET"],
      "AllowedHeaders": ["*"],
      "MaxAgeSeconds": 3000
    }]
  }'
```

✅ **Single quotes are safe here**

— the CORS JSON contains no shell variables, so single quotes work fine. The bucket name is passed via `--bucket $INPUT_BUCKET` (outside the quotes) which does expand correctly. Single quotes are only a problem when variables appear INSIDE the quoted JSON string.

WHY DOES THE INPUT BUCKET NEED CORS?

The upload flow works like this: browser calls API Gateway → gets a pre-signed S3 URL → browser then PUTs the file directly to S3 using that URL. This direct S3 PUT comes from the user's browser (a different origin than S3). Browsers enforce CORS — they send a preflight OPTIONS request before every cross-origin PUT. Without CORS configured on the S3 bucket, S3 returns no CORS headers and the browser blocks the upload before it even starts. The actual file never reaches S3.

5 Enable CORS on the output bucket (browsers need this for direct GET)

```
aws s3api put-bucket-cors \
  --bucket $OUTPUT_BUCKET \
```

```
  --region ap-south-1 \
  --cors-configuration '{
    "CORSRules": [{
      "AllowedOrigins": ["*"],
      "AllowedMethods": ["GET"],
      "AllowedHeaders": ["*"],
      "MaxAgeSeconds": 3000
    }]
  }'
```

> **WHY DOES THE OUTPUT BUCKET ALSO NEED CORS?**
>
> Same reason — when the user clicks "Download", the browser follows the pre-signed URL directly to S3 to fetch the processed image. This is a cross-origin GET from the user's browser. Without CORS on the output bucket, the download button works in some browsers (those that don't enforce CORS on GETs) but fails in others with a CORS error.

5 Phase 2 — Create IAM Role for Lambda

Why before Lambda? When you run `aws lambda create-function` you must supply a `--role` ARN at creation time. The role must exist before the function is created. All 3 Lambda functions will share this single role.

1 Create the IAM role with Lambda trust policy

```
aws iam create-role \
  --role-name Lambda-Image-Processing-Role \
  --assume-role-policy-document '{
    "Version": "2012-10-17",
    "Statement": [{
      "Effect": "Allow",
      "Principal": {"Service": "lambda.amazonaws.com"},
      "Action": "sts:AssumeRole"
    }]
  }'
```

> **WHAT IS THE TRUST POLICY AND WHY DOES IT MATTER?**
>
> An IAM Role by itself grants permissions — but you also need to specify WHO is allowed to use (assume) the role. This "trust policy" is a separate document that says "the Lambda service (lambda.amazonaws.com) is allowed to assume this role." Without it, Lambda cannot pick up the role's permissions when your function runs. This is a frequent point of confusion: permissions policy = what the role CAN do; trust policy = WHO can use the role.

Issue Hit: EntityAlreadyExists — Role already exists

```
An error occurred (EntityAlreadyExists) when calling the CreateRole operation:
Role with name Lambda-Image-Processing-Role already exists.
```

Why it happened:

The IAM role was created during a previous deployment and still exists in your AWS account. IAM roles are global (not region-specific) and persist until explicitly deleted. Unlike S3 buckets where you needed a new suffix, IAM roles with the same name across deployments is fine — you can simply reuse the existing one.

Fix — skip create-role and just fetch the existing ARN:

```
ROLE_ARN=$(aws iam get-role --role-name Lambda-Image-Processing-Role \
  --query 'Role.Arn' --output text)
echo $ROLE_ARN
```

Then continue with Steps 2 and 3 (attaching policies). The `attach-role-policy` and `put-role-policy` commands are safe to re-run on an existing role — they are idempotent (attaching an already-attached policy just returns success silently).

Remember for re-deployments:

IAM roles, policies, CloudWatch log groups, and Lambda functions all persist between deployments. Only S3 objects are truly ephemeral. The pattern is: try to create, if EntityAlreadyExists get the existing resource ARN and continue.

2 Attach CloudWatch Logs permission (built-in AWS policy)

```
aws iam attach-role-policy \
  --role-name Lambda-Image-Processing-Role \
  --policy-arn arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole
```

WHAT DOES AWSLAMBDABASICEXECUTIONROLE GRANT?

This AWS-managed policy grants permission to create CloudWatch log groups, log streams, and write log events. Without it, your Lambda functions run but you cannot see any print() output or error tracebacks in CloudWatch Logs — debugging becomes impossible. This is the minimum policy every Lambda function needs.

3 Attach S3 read/write permissions (custom inline policy)

```
# IMPORTANT: Use double quotes + escaped inner quotes so $INPUT_BUCKET and $OUTPUT_BUCKET expan
aws iam put-role-policy \
  --role-name Lambda-Image-Processing-Role \
  --policy-name S3ImagePipelineAccess \
  --policy-document "{
```

```
  \"Version\": \"2012-10-17\",
  \"Statement\": [
    {
      \"Effect\": \"Allow\",
      \"Action\": [\"s3:GetObject\", \"s3:PutObject\", \"s3:HeadObject\"],
      \"Resource\": [
        \"arn:aws:s3:::${INPUT_BUCKET}/*\",
        \"arn:aws:s3:::${OUTPUT_BUCKET}/*\"
      ]
    },
    {
      \"Effect\": \"Allow\",
      \"Action\": \"s3:GeneratePresignedUrl\",
      \"Resource\": \"*\"
    }
  ]
}"
```

⚠**Same single-quote vs double-quote rule applies here:**

`${INPUT_BUCKET}` and `${OUTPUT_BUCKET}` appear inside the JSON string. Use double quotes on the outer string and escape all inner JSON quotes with backslashes. If you use single quotes, the Resource ARNs will contain the literal text `$INPUT_BUCKET` and the policy will silently apply to a bucket that does not exist.

BREAKING DOWN THE S3 PERMISSIONS NEEDED

s3:GetObject on input bucket: Lambda 2 (ImageProcessor) downloads the original image from the input bucket to /tmp for processing.

s3:PutObject on output bucket: Lambda 2 uploads 3 processed images to the output bucket after resizing/compressing.

s3:HeadObject: Lambda 3 (get-processed-images) may check if the processed files exist before generating download URLs.

s3:GeneratePresignedUrl on Resource: *: Pre-signed URL generation is a client-side SDK operation — it uses your IAM credentials to sign a URL that can be used by anyone. It technically doesn't need a specific resource because no S3 API call is made — the SDK signs the URL locally using your credentials. However the effective S3 operation (PUT/GET) is still bound by the bucket policies above.

✅**Save the role ARN — you need it in Phase 3:**

```
ROLE_ARN=$(aws iam get-role --role-name Lambda-Image-Processing-Role \
  --query 'Role.Arn' --output text)
echo $ROLE_ARN
# arn:aws:iam::264449293739:role/Lambda-Image-Processing-Role
```

6. Phase 3 — Deploy 3 Lambda Functions

6 Phase 3 — Deploy 3 Lambda Functions

Why after IAM? Lambda create-function requires the role ARN at creation time. Why before API Gateway (Phase 5)? API Gateway integrations need the Lambda function ARNs. Why before S3 event trigger (Phase 4)? The S3 trigger notification points to a specific Lambda ARN — the function must exist first.

⚠ **Critical difference from LinkForge:**

Lambda 2 (ImageProcessor) uses Pillow which is NOT available in the Lambda runtime. You MUST bundle Pillow into the ZIP. Lambda 1 and 3 have no external dependencies — only boto3, which IS pre-installed. Two different packaging approaches for this one project.

1 Package and deploy Lambda 1 — generate-upload-url (no dependencies)

```
cd lambda-functions/generate-upload-url
zip ../../build/generate_upload_url.zip lambda_function.py
cd ../..

aws lambda create-function \
  --function-name generate-upload-url \
  --runtime python3.11 \
  --role $ROLE_ARN \
  --handler lambda_function.lambda_handler \
  --zip-file fileb://build/generate_upload_url.zip \
  --timeout 10 \
  --memory-size 128 \
  --region $REGION \
  --environment Variables='{
    "INPUT_BUCKET": "$INPUT_BUCKET"
  }'
```

WHAT THIS FUNCTION DOES AND WHY IT'S SIMPLE TO PACKAGE

This function receives `{"filename": "photo.jpg", "contentType": "image/jpeg"}`, calls `s3.generate_presigned_url("put_object", ...)`, and returns the signed URL. It only uses boto3 which is pre-installed in every Lambda Python runtime — no pip install needed, the .py file alone is the entire deployment package.

`--timeout 10`: Pre-signed URL generation is instant (no network call — it's a local crypto signing operation). 10 seconds is generous but appropriate for a URL generation endpoint.

`--memory-size 128`: Minimum memory. URL generation is trivial computation — no reason to pay for more.

2 Package and deploy Lambda 3 — get-processed-images (no dependencies)

```
mkdir -p build
cd lambda-functions/get-processed-images
```

```
zip ../../build/get_processed_images.zip lambda_function.py
cd ../..

aws lambda create-function \
  --function-name get-processed-images \
  --runtime python3.11 \
  --role $ROLE_ARN \
  --handler lambda_function.lambda_handler \
  --zip-file fileb://build/get_processed_images.zip \
  --timeout 10 \
  --memory-size 128 \
  --region $REGION \
  --environment Variables='{
    "OUTPUT_BUCKET": "$OUTPUT_BUCKET"
  }'
```

WHAT THIS FUNCTION DOES

Receives a filename (e.g. photo.jpg), generates three pre-signed S3 GET URLs pointing to 1080p/photo.jpg, 720p/photo.jpg, 480p/photo.jpg in the output bucket, and returns them as a JSON object. Like Lambda 1, it only uses boto3 — simple packaging. 1-hour URL expiry (3600 seconds) gives the user time to click all three download links without them expiring.

3 Package Lambda 2 — ImageProcessor (Pillow must be bundled)

```
# Create a clean packaging directory
mkdir -p build/image-processor-pkg

# Install Pillow into that directory (this is the critical step)
pip3 install pillow --target build/image-processor-pkg --platform manylinux2014_x86_64 \
  --implementation cp --python-version 3.11 --only-binary=:all:

# Copy the Lambda code into the same directory
cp lambda-functions/image-processor/lambda_function.py build/image-processor-pkg/

# Zip everything together — Pillow + lambda_function.py at the root of the ZIP
cd build/image-processor-pkg
zip -r ../image_processor.zip .
cd ../..
```

Part	What it does and why exactly this way
`pip3 install pillow --target`	Installs Pillow into a specific directory instead of the system Python. --target means "put the package files here" — this is the standard way to create Lambda deployment packages with dependencies.
`--platform manylinux2014_x86_64`	CRITICAL: Pillow is a C extension library. Your Mac/Windows has a different binary format than the Lambda Linux runtime. This flag forces pip to download the Linux-compatible binary wheel instead of the binary for your local machine. Without this, Pillow will import successfully on your Mac but crash

	with ImportError on Lambda.
`--implementation cp --python-version 3.11`	Specifies the exact CPython 3.11 binary to download — must match the Lambda runtime (python3.11). This ensures the Pillow C extensions are compiled for the right Python ABI.
`--only-binary=:all:`	Forces pip to only download pre-built wheel files (no compilation from source). Without this, pip might try to compile Pillow from source code, which requires build tools not available in the sandboxed environment.
`zip -r ../image_processor.zip .`	Zips everything in the packaging directory — both lambda_function.py and the entire Pillow package directory tree — into a single ZIP. Lambda extracts this ZIP into the execution environment, making Pillow importable by your code.

Most common packaging mistake: missing the Linux platform flag

```
# WRONG — installs Pillow for your local OS (Mac/Windows)
pip3 install pillow --target build/image-processor-pkg

# CORRECT — installs the Linux binary that Lambda can run
pip3 install pillow --target build/image-processor-pkg \
  --platform manylinux2014_x86_64 \
  --implementation cp --python-version 3.11 --only-binary=:all:
```

Symptom: Lambda deploys fine, but when triggered it throws `ImportError: cannot import name '_imaging' from 'PIL'`. This means the Pillow binary is for the wrong OS. Re-package with the correct platform flag.

4 Deploy Lambda 2 — ImageProcessor

```
aws lambda create-function \
  --function-name ImageProcessorLambda-Shivam \
  --runtime python3.11 \
  --role $ROLE_ARN \
  --handler lambda_function.lambda_handler \
  --zip-file fileb://build/image_processor.zip \
  --timeout 59 \
  --memory-size 1024 \
  --region $REGION \
  --environment Variables='{
    "OUTPUT_BUCKET": "$OUTPUT_BUCKET",
    "COMPRESSION_QUALITY": "85"
  }'
```

WHY TIMEOUT 59 AND MEMORY 1024 — VERY DIFFERENT FROM LAMBDAS 1 AND 3

Timeout 59 seconds: Lambda has a hard maximum of 15 minutes, but a processing function

needs careful tuning. A 14MB image takes ~2.5 seconds to download, process at 3 resolutions, and upload 3 files. A 50MB image near the limit takes ~6 seconds. Setting 59 seconds gives ample buffer while keeping the function well within Lambda's free tier duration billing. Setting it to the Lambda maximum (900s) would be wasteful — if there's a bug causing an infinite loop, you'd pay for 15 minutes instead of failing fast at 59 seconds.

Memory 1024 MB: Pillow loads the entire image into memory for processing. A 14MB JPEG decompressed into raw pixels becomes approximately 100–200 MB in memory. Processing 3 resolutions simultaneously means peak memory usage can hit 300–400 MB. 1024 MB gives a safe ceiling. Important: Lambda's CPU allocation is directly proportional to memory — more memory = more CPU = faster processing. The image processor benefits from more CPU to run Pillow's compression algorithms faster.

KEY INSIGHT — THIS LAMBDA IS NEVER CALLED BY API GATEWAY

Notice there is no API Gateway route pointing to ImageProcessorLambda-Shivam. It is triggered exclusively by the S3 event configured in Phase 4. When a user uploads an image (via the pre-signed URL from Lambda 1), the image lands in the input bucket, and S3 automatically fires an event that invokes this Lambda. The user never knows this Lambda exists — they just wait ~5 seconds and then request download URLs from Lambda 3.

5 Update Lambda code after initial deployment (re-deploy pattern)

```
# For Lambdas 1 and 3 (no dependencies — just re-zip the .py file):
cd lambda-functions/generate-upload-url
zip ../../build/generate_upload_url.zip lambda_function.py
cd ../..
aws lambda update-function-code \
  --function-name generate-upload-url \
  --zip-file fileb://build/generate_upload_url.zip \
  --region $REGION

# For Lambda 2 (dependencies — must re-install Pillow and re-zip):
pip3 install pillow --target build/image-processor-pkg \
  --platform manylinux2014_x86_64 \
  --implementation cp --python-version 3.11 --only-binary=:all:
cp lambda-functions/image-processor/lambda_function.py build/image-processor-pkg/
cd build/image-processor-pkg && zip -r ../image_processor.zip . && cd ../..
aws lambda update-function-code \
  --function-name ImageProcessorLambda-Shivam \
  --zip-file fileb://build/image_processor.zip \
  --region $REGION
```

WHY UPDATE-FUNCTION-CODE INSTEAD OF CREATE-FUNCTION ON RE-DEPLOY?

`create-function` fails if the function already exists. `update-function-code` replaces only the code ZIP without touching the function's configuration (timeout, memory, env vars, IAM role). Use this for all subsequent code changes after the initial deployment.

7 Phase 4 — S3 Event Trigger (Wires S3 to ImageProcessor)

Why Phase 4 after Lambda? The S3 event notification points to a specific Lambda ARN. The Lambda must exist before S3 can be configured to call it. Similarly, you must grant Lambda invoke permission before adding the notification — otherwise S3 cannot deliver the event.

This is the "magic" that makes the pipeline automatic:
Once this trigger is configured, every single image uploaded to the input bucket (regardless of how it got there — pre-signed URL, CLI, console) automatically triggers the ImageProcessor Lambda. No polling, no manual invocation. S3 pushes an event synchronously to Lambda.

1 Grant S3 permission to invoke the ImageProcessor Lambda

```
aws lambda add-permission \
  --function-name ImageProcessorLambda-Shivam \
  --statement-id s3-trigger-permission \
  --action lambda:InvokeFunction \
  --principal s3.amazonaws.com \
  --source-arn arn:aws:s3:::$INPUT_BUCKET \
  --source-account $ACCOUNT_ID \
  --region $REGION
```

Flag	What it does and why
`--principal s3.amazonaws.com`	Grants S3 (not API Gateway, not a user) permission to invoke this Lambda. S3 uses its own service principal — different from the apigateway.amazonaws.com used in LinkForge.
`--source-arn arn:aws:s3:::$INPUT_BUCKET`	Restricts to ONLY the input bucket. Without this, any S3 bucket in your account (or even other accounts) could invoke the ImageProcessor — a security risk. Scoping to one bucket ARN ensures only uploads to this specific bucket trigger processing.
`--source-account $ACCOUNT_ID`	Additional security restriction — only events from your own account. Prevents "confused deputy" attacks where a malicious bucket in another account could trigger your Lambda.

WHY MUST THIS PERMISSION BE ADDED BEFORE THE S3 NOTIFICATION?

When you configure the S3 bucket notification in Step 2, AWS validates that the target Lambda function actually allows S3 to invoke it. If the permission doesn't exist, the put-bucket-notification-configuration command itself fails with "Unable to validate the following destination configurations." Always add the Lambda permission first, then configure the S3 notification.

2 Configure the S3 bucket notification

```
PROCESSOR_ARN="arn:aws:lambda:$REGION:$ACCOUNT_ID:function:ImageProcessorLambda-Shivam"

aws s3api put-bucket-notification-configuration \
  --bucket $INPUT_BUCKET \
  --region $REGION \
  --notification-configuration "{
    \"LambdaFunctionConfigurations\": [{
      \"LambdaFunctionArn\": \"$PROCESSOR_ARN\",
      \"Events\": [\"s3:ObjectCreated:*\"]
    }]
  }"
```

WHAT DOES S3:OBJECTCREATED:* MEAN?

`s3:ObjectCreated:*` is a wildcard that matches any object creation event — this includes `s3:ObjectCreated:Put` (direct PUT), `s3:ObjectCreated:Post` (multipart upload), `s3:ObjectCreated:Copy` (object copy), and `s3:ObjectCreated:CompleteMultipartUpload`. Using * ensures the ImageProcessor fires regardless of how the image arrived in the bucket. If you used Put only, a large image uploaded via multipart would not trigger processing.

The event payload S3 sends to Lambda includes the bucket name, object key, object size, and ETag. The ImageProcessor Lambda reads these from `event['Records'][0]['s3']`.

8 Phase 5 — API Gateway REST API (v1)

Why Phase 5 after Lambda? API Gateway integrations reference Lambda function ARNs. The functions must exist to be referenced. Why last before frontend? The frontend JavaScript needs the API Gateway URL — which is only known after the API is deployed to a stage.

REST API V1 VS HTTP API V2 — THIS PROJECT USES V1

LinkForge used HTTP API (v2) with the `aws apigatewayv2` CLI command. This project uses REST API (v1) with the `aws apigateway` CLI command. The APIs are different products with different CLI interfaces. Key differences: v1 (REST) requires explicit deployment to a stage after each change; v2 (HTTP) has AutoDeploy. v1 requires manual CORS configuration with OPTIONS methods; v2 has built-in CORS settings. v1 is older and more feature-rich; v2 is simpler and cheaper.

1 Create the REST API

```
API_ID=$(aws apigateway create-rest-api \
  --name "ImagePipelineAPI" \
  --description "API for Image Processing Pipeline" \
  --endpoint-configuration types=REGIONAL \
  --region $REGION \
  --query 'id' --output text)

echo "API ID: $API_ID"

# Get the root resource ID (needed to create child resources)
ROOT_ID=$(aws apigateway get-resources \
  --rest-api-id $API_ID \
  --region $REGION \
  --query 'items[?path==`/`].id' --output text)

echo "Root Resource ID: $ROOT_ID"
```

WHAT IS THE ROOT RESOURCE AND WHY DO YOU NEED ITS ID?

Every REST API in API Gateway has a root resource "/" that represents the base URL. When you create child resources (like /generate-upload-url or /processed-images), you must specify the parent resource they live under. The root resource ID is the parent for both your endpoints. Every API Gateway resource has a unique ID — you need it for subsequent CLI commands to create methods and integrations.

`--endpoint-configuration types=REGIONAL`: Creates a regional API (served from the ap-south-1 region only, not from CloudFront edge locations). Regional endpoints are faster for users in the same region and have lower costs than EDGE endpoints.

Issue Hit: zsh: number expected — when running the API creation command

```
API ID: 6vk04wd8h5
zsh: number expected
Root Resource ID: vmtbkp014i
```

Why it happened:

The `--query 'items[?path==`/`].id'` part of the ROOT_ID command uses backtick characters inside single-quoted JMESPath. zsh misreads the backtick characters in the query string as a shell arithmetic expression and throws "number expected." This is a zsh-specific parsing quirk — not a bash issue.

This is harmless — both values came back correctly.

The API was created and both variables are set. You can verify:

```
echo "API ID: $API_ID"
echo "Root ID: $ROOT_ID"
```

If either variable is empty, re-run that specific sub-command. The zsh error does not abort the command — it just warns and continues.

Prevention (if you want to avoid the warning):

Use escaped backticks or double quotes around the query:

```
ROOT_ID=$(aws apigateway get-resources \
  --rest-api-id $API_ID \
  --region $REGION \
  --query "items[?path=='/'].id" --output text)
```

2 Create /generate-upload-url endpoint with CORS

```
# Create the resource (URL path)
UPLOAD_URL_RESOURCE=$(aws apigateway create-resource \
  --rest-api-id $API_ID \
  --parent-id $ROOT_ID \
  --path-part "generate-upload-url" \
  --region $REGION \
  --query 'id' --output text)

# Add POST method (no auth required — public endpoint)
aws apigateway put-method \
  --rest-api-id $API_ID \
  --resource-id $UPLOAD_URL_RESOURCE \
  --http-method POST \
  --authorization-type NONE \
  --region $REGION

# Integrate POST with Lambda 1
LAMBDA1_ARN="arn:aws:lambda:$REGION:$ACCOUNT_ID:function:generate-upload-url"

aws apigateway put-integration \
  --rest-api-id $API_ID \
  --resource-id $UPLOAD_URL_RESOURCE \
  --http-method POST \
  --type AWS_PROXY \
  --integration-http-method POST \
  --uri "arn:aws:apigateway:$REGION:lambda:path/2015-03-31/functions/$LAMBDA1_ARN/invocations"
  --region $REGION

# Grant API Gateway permission to invoke Lambda 1
aws lambda add-permission \
  --function-name generate-upload-url \
  --statement-id apigateway-invoke-upload \
  --action lambda:InvokeFunction \
  --principal apigateway.amazonaws.com \
  --source-arn "arn:aws:execute-api:$REGION:$ACCOUNT_ID:$API_ID/*/*" \
  --region $REGION

# Add CORS — OPTIONS method with MOCK integration
aws apigateway put-method \
  --rest-api-id $API_ID \
  --resource-id $UPLOAD_URL_RESOURCE \
  --http-method OPTIONS \
  --authorization-type NONE \
  --region $REGION

aws apigateway put-integration \
```

```
  --rest-api-id $API_ID \
  --resource-id $UPLOAD_URL_RESOURCE \
  --http-method OPTIONS \
  --type MOCK \
  --request-templates '{"application/json": "{\"statusCode\": 200}"}' \
  --region $REGION

aws apigateway put-method-response \
  --rest-api-id $API_ID \
  --resource-id $UPLOAD_URL_RESOURCE \
  --http-method OPTIONS \
  --status-code 200 \
  --response-parameters '{
    "method.response.header.Access-Control-Allow-Headers": false,
    "method.response.header.Access-Control-Allow-Methods": false,
    "method.response.header.Access-Control-Allow-Origin": false
  }' \
  --region $REGION

aws apigateway put-integration-response \
  --rest-api-id $API_ID \
  --resource-id $UPLOAD_URL_RESOURCE \
  --http-method OPTIONS \
  --status-code 200 \
  --response-parameters '{
    "method.response.header.Access-Control-Allow-Headers": "'"'"'Content-Type,X-Amz-Date,Author
    "method.response.header.Access-Control-Allow-Methods": "'"'"'POST,OPTIONS'"'"'",
    "method.response.header.Access-Control-Allow-Origin": "'"'"'*'"'"'"
  }' \
  --region $REGION
```

WHY DOES REST API NEED MANUAL CORS SETUP WITH OPTIONS (UNLIKE HTTP API V2)?

HTTP API (v2) has a `CorsConfiguration` block that handles everything automatically. REST API (v1) has no such block — you must manually create an OPTIONS method on every resource and configure it with a MOCK integration (a fake endpoint that returns CORS headers without calling Lambda). The MOCK integration returns a 200 with the three CORS headers browsers need: Allow-Origin, Allow-Methods, Allow-Headers. This is the "preflight response" that browsers check before sending the actual POST.

The Lambda functions also return `Access-Control-Allow-Origin: *` in their response headers — this handles the "actual response" CORS check. You need CORS on both the OPTIONS preflight AND the actual response.

Issue Hit: BadRequestException — AWS ARN for integration must contain path or action

```
An error occurred (BadRequestException) when calling the PutIntegration operation:
AWS ARN for integration must contain path or action
```

Why it happened:

The `put-integration` command for the POST method failed because the `--uri` ARN was malform
`$LAMBDA1_ARN` variable was not expanded before being embedded in the URI string. The URI must
format:
`arn:aws:apigateway:REGION:lambda:path/2015-03-31/functions/FULL_LAMBDA_ARN/invocations`

Everything else in the step succeeded

— the POST method was created, Lambda permission was granted, and the OPTIONS MOCK for CO
went through. Only the `put-integration` for POST failed.

Fix — build the Lambda ARN explicitly and verify it before using:

```
# Build and verify the ARN first
LAMBDA1_ARN="arn:aws:lambda:${REGION}:${ACCOUNT_ID}:function:generate-upload-url"
echo $LAMBDA1_ARN   # ← always verify this looks correct before the next command

# Then run put-integration separately using ${} braces for safe expansion
aws apigateway put-integration \
  --rest-api-id $API_ID \
  --resource-id $UPLOAD_URL_RESOURCE \
  --http-method POST \
  --type AWS_PROXY \
  --integration-http-method POST \
  --uri "arn:aws:apigateway:${REGION}:lambda:path/2015-03-31/functions/${LAMBDA1_ARN}/invocat
  --region $REGION
```

Success response:

`{"type": "AWS_PROXY", "httpMethod": "POST", "uri": "arn:aws:apigateway:ap-south-1:lambda:path/2
31/functions/arn:aws:lambda:..."}` — confirms the integration is wired correctly.

Same issue appears in Step 3

for the GET method on `/processed-images/{filename}`. Apply the same fix with `LAMBDA3_ARN`.

WHAT IS AWS_PROXY INTEGRATION TYPE?

AWS_PROXY means API Gateway passes the entire HTTP request to Lambda as a JSON event — headers, body, path parameters, query parameters all included. Lambda returns a JSON response object with statusCode, headers, and body, and API Gateway transforms it back into an HTTP response. This is the standard modern pattern. The alternative (AWS integration type without PROXY) requires you to manually define request/response mapping templates — much more complex and rarely needed.

3 Create /processed-images/{filename} endpoint with CORS

```
# Create /processed-images parent resource
PROC_IMAGES_RESOURCE=$(aws apigateway create-resource \
  --rest-api-id $API_ID \
  --parent-id $ROOT_ID \
  --path-part "processed-images" \
  --region $REGION \
  --query 'id' --output text)

# Create {filename} path parameter child resource
FILENAME_RESOURCE=$(aws apigateway create-resource \
  --rest-api-id $API_ID \
  --parent-id $PROC_IMAGES_RESOURCE \
  --path-part "{filename}" \
```

```
  --region $REGION \
  --query 'id' --output text)

# Add GET method with filename path parameter required
aws apigateway put-method \
  --rest-api-id $API_ID \
  --resource-id $FILENAME_RESOURCE \
  --http-method GET \
  --authorization-type NONE \
  --request-parameters "method.request.path.filename=true" \
  --region $REGION

# Integrate GET with Lambda 3
LAMBDA3_ARN="arn:aws:lambda:$REGION:$ACCOUNT_ID:function:get-processed-images"

aws apigateway put-integration \
  --rest-api-id $API_ID \
  --resource-id $FILENAME_RESOURCE \
  --http-method GET \
  --type AWS_PROXY \
  --integration-http-method POST \
  --uri "arn:aws:apigateway:$REGION:lambda:path/2015-03-31/functions/$LAMBDA3_ARN/invocations"
  --region $REGION

# Grant API Gateway permission to invoke Lambda 3
aws lambda add-permission \
  --function-name get-processed-images \
  --statement-id apigateway-invoke-processed \
  --action lambda:InvokeFunction \
  --principal apigateway.amazonaws.com \
  --source-arn "arn:aws:execute-api:$REGION:$ACCOUNT_ID:$API_ID/*/*" \
  --region $REGION

# Add CORS OPTIONS to the {filename} resource (same pattern as above)
aws apigateway put-method \
  --rest-api-id $API_ID \
  --resource-id $FILENAME_RESOURCE \
  --http-method OPTIONS \
  --authorization-type NONE \
  --region $REGION

aws apigateway put-integration \
  --rest-api-id $API_ID \
  --resource-id $FILENAME_RESOURCE \
  --http-method OPTIONS \
  --type MOCK \
  --request-templates '{"application/json": "{\"statusCode\": 200}"}' \
  --region $REGION

aws apigateway put-method-response \
  --rest-api-id $API_ID \
  --resource-id $FILENAME_RESOURCE \
  --http-method OPTIONS \
  --status-code 200 \
  --response-parameters '{
    "method.response.header.Access-Control-Allow-Headers": false,
    "method.response.header.Access-Control-Allow-Methods": false,
    "method.response.header.Access-Control-Allow-Origin": false
  }' \
  --region $REGION
```

```
aws apigateway put-integration-response \
  --rest-api-id $API_ID \
  --resource-id $FILENAME_RESOURCE \
  --http-method OPTIONS \
  --status-code 200 \
  --response-parameters '{
    "method.response.header.Access-Control-Allow-Headers": "'"'"'Content-Type,X-Amz-Date,Author
    "method.response.header.Access-Control-Allow-Methods": "'"'"'GET,OPTIONS'"'"'",
    "method.response.header.Access-Control-Allow-Origin": "'"'"'*'"'"'"
  }' \
  --region $REGION
```

WHY IS THE PATH-PART "{FILENAME}" IN CURLY BRACES?

Curly braces denote a path parameter in API Gateway. When a request comes in for `/processed-images/photo.jpg`, API Gateway captures "photo.jpg" as the value of the `filename` path parameter and passes it to Lambda in `event["pathParameters"]["filename"]`. Without curly braces, API Gateway treats it as a literal string — only requests to exactly `/processed-images/{filename}` would match, which is not useful.

4 Deploy the API to a stage — REST API requires explicit deployment

```
aws apigateway create-deployment \
  --rest-api-id $API_ID \
  --stage-name prod \
  --stage-description "Production Stage" \
  --description "Initial deployment" \
  --region $REGION

# Your API base URL is now:
echo "API URL: https://$API_ID.execute-api.$REGION.amazonaws.com/prod"
```

WHY DOES REST API (V1) NEED AN EXPLICIT DEPLOYMENT STEP?

REST API in API Gateway uses a "deployment + stage" model. A deployment is a snapshot of your API configuration at a point in time. A stage (prod, dev, staging) points to a specific deployment. This design means you can have different stages point to different versions of your API — useful for blue/green deployments. The downside: every time you change a method, integration, or CORS setting, you must create a new deployment and it takes effect in the stage only after the deployment. This is different from HTTP API v2 which has AutoDeploy — changes take effect immediately.

If you make any change to the API after this step, you MUST run create-deployment again. Forgetting this is the most common source of confusion in API Gateway v1 — "I changed the Lambda but the API still behaves the same."

9 Phase 6 — Deploy Frontend to S3

Why last? The frontend JavaScript (app.js) contains the API Gateway URL. That URL is only known after Phase 5 deploys the API. You must update the URL in the code before uploading the files.

1 Update the API URL in app.js

```
# Get your API URL
echo "https://$API_ID.execute-api.$REGION.amazonaws.com/prod"
```

Open `frontend/app.js` and update line 2:

```
// Change this line:
const API_BASE_URL = 'https://38hmfok3uk.execute-api.ap-south-1.amazonaws.com/prod';
// To your actual API ID from Phase 5:
const API_BASE_URL = 'https://<YOUR_API_ID>.execute-api.ap-south-1.amazonaws.com/prod';
```

> **WHY IS THE URL HARDCODED IN JAVASCRIPT (NOT INJECTED AT RUNTIME)?**
>
> S3 static website hosting serves flat files — there is no server that can inject values at request time. The API URL must be baked into the JavaScript file before it's uploaded. This is the same limitation as LinkForge. In production systems, this would be handled by a build step (e.g. webpack with environment variables), but for this project, manual editing is the approach.

2 Sync frontend files to S3

```
# Make sure you are inside the project root (serverless-image-optimizer/)
# before running this — the frontend/ folder must exist relative to your cwd
aws s3 sync frontend/ s3://$FRONTEND_BUCKET/ \
  --region $REGION \
  --delete \
  --exclude "README.md"

# Set correct content types
aws s3 cp s3://$FRONTEND_BUCKET/ s3://$FRONTEND_BUCKET/ \
  --recursive --exclude "*" --include "*.html" \
  --content-type "text/html" --metadata-directive REPLACE --region $REGION

aws s3 cp s3://$FRONTEND_BUCKET/ s3://$FRONTEND_BUCKET/ \
  --recursive --exclude "*" --include "*.js" \
  --content-type "application/javascript" --metadata-directive REPLACE --region $REGION

aws s3 cp s3://$FRONTEND_BUCKET/ s3://$FRONTEND_BUCKET/ \
  --recursive --exclude "*" --include "*.css" \
  --content-type "text/css" --metadata-directive REPLACE --region $REGION
```

Issue 1 Hit: "The user-provided path frontend/ does not exist"

```
fatal error: The user-provided path frontend/ does not exist.
```

Why it happened:

The terminal was in a parent directory (`Image Optimization Pipeline ReDeploy/`) that contained th project folder as a subdirectory, rather than inside the project root itself. The `frontend/` folder onl exists relative to the project root.

Fix — always verify your working directory before running S3 sync:

```
pwd           # confirm you are in serverless-image-optimizer/
ls            # confirm frontend/ folder is visible in the listing
aws s3 sync frontend/ s3://$FRONTEND_BUCKET/ --region $REGION --delete --exclude "README.md"
```

If you see the project folder listed (not its contents), you need to `cd serverless-image-optimizer` first.

Issue 2 Hit: Invalid bucket name "" — $FRONTEND_BUCKET is empty

```
fatal error: Parameter validation failed:
Invalid bucket name "": Bucket name must match the regex...
```

Why it happened:

Shell variables are not persistent across terminal sessions or directory changes. When you opened new terminal tab or the variable was never exported in that session, `$FRONTEND_BUCKET` resolved tc an empty string. The `s3 sync` command then tried to sync to a bucket named `""`.

Fix — always re-export variables at the start of every terminal session before running any command:

```
export SUFFIX="anshuman"
export REGION="ap-south-1"
export ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
export INPUT_BUCKET="image-pipeline-input-$SUFFIX"
export OUTPUT_BUCKET="image-pipeline-output-$SUFFIX"
export FRONTEND_BUCKET="image-pipeline-frontend-$SUFFIX"
# Verify before continuing:
echo "$INPUT_BUCKET | $OUTPUT_BUCKET | $FRONTEND_BUCKET"
```

Pro tip for interviews:

Keep this export block at the top of a scratch file. Copy-paste it at the start of every terminal session. Never assume variables from a previous session are still set.

WHY SEPARATELY FIX CONTENT TYPES AFTER SYNC?

`aws s3 sync` sometimes guesses MIME types incorrectly or uses `application/octet-stream` as a fallback. Browsers use the `Content-Type` HTTP header to decide how to handle a file. If .js files have the wrong content type, modern browsers may refuse to execute them (especially with strict CSP settings). The `cp` commands with `--metadata-directive REPLACE` force-overwrite the content type on already-uploaded files without re-uploading the data.

3 Get your live website URL

```
echo "Frontend: http://$FRONTEND_BUCKET.s3-website.$REGION.amazonaws.com"
echo "API: https://$API_ID.execute-api.$REGION.amazonaws.com/prod"
```

WHY HTTP AND NOT HTTPS FOR THE FRONTEND?

S3 static website hosting only supports HTTP. Adding HTTPS requires putting CloudFront (AWS CDN) in front of S3 with an ACM certificate — that's on the future enhancements list. For an interview demo, HTTP is fine. Note: some browsers may warn about a "mixed content" issue if the frontend (HTTP) calls an API Gateway (HTTPS) — but modern browsers generally allow this direction (HTTP page → HTTPS API). The reverse (HTTPS page → HTTP API) would be blocked.

10 Verification & Testing — Layer by Layer

TEST ORDER MATTERS — BOTTOM UP

Test each layer independently before testing the layer above it. If the upload API fails, don't test the UI. If the ImageProcessor doesn't trigger, don't test the download API. Isolating failures layer by layer tells you exactly where to fix, not just that something is broken.

1 Test Lambda 1 — generate-upload-url

```
API_BASE="https://$API_ID.execute-api.$REGION.amazonaws.com/prod"

curl -X POST $API_BASE/generate-upload-url \
  -H "Content-Type: application/json" \
  -d '{"filename": "test.jpg", "contentType": "image/jpeg"}' \
  | python3 -m json.tool
```

EXPECTED RESPONSE AND WHAT IT TELLS YOU

Expected: `{"uploadUrl": "https://image-pipeline-input-$SUFFIX.s3.amazonaws.com/test.jpg?X-Amz-...", "filename": "test.jpg"}`

This confirms: API Gateway route exists, Lambda 1 was invoked, IAM role allows pre-signed URL generation, and the URL is for the correct input bucket. If you get `{"message": "Internal Server Error"}` check CloudWatch Logs for Lambda 1. If you get `{"message": "Missing Authentication Token"}` the API stage was not deployed — run create-deployment again.

Critical check: verify the bucket name in the uploadUrl matches YOUR bucket — the URL should say `image-pipeline-input-anshuman`, not `image-pipeline-input-sid`.

Issue Hit: Upload fails with 403 — pre-signed URL pointing to wrong bucket

```
# curl response showed:
"uploadUrl": "https://image-pipeline-input-sid.s3.amazonaws.com/test.jpg?..."
#                                            ^^^
#                         Should be: image-pipeline-input-anshuman

# Browser error:
Failed to load resource: the server responded with a status of 403 (Forbidden)
Error: Failed to upload image to S3
```

Root cause — the bucket name was hardcoded in the Lambda source code, not read from the environment variable:

```
# WRONG — what the original lambda_function.py had:
s3 = boto3.client("s3")
def lambda_handler(event, context):
    url = s3.generate_presigned_url("put_object",
        Params={"Bucket": "image-pipeline-input-sid", ...})  # ← HARDCODED

# CORRECT — reads from env var (fix applied):
import os
BUCKET = os.environ.get("INPUT_BUCKET", "image-pipeline-input-sid")
def lambda_handler(event, context):
    url = s3.generate_presigned_url("put_object",
        Params={"Bucket": BUCKET, ...})  # ← reads env var
```

Why setting the env var with update-function-configuration was not enough:

Setting an environment variable only helps if the code actually reads it with `os.environ.get()`. The original generate-upload-url and get-processed-images code had the bucket names as string literals — `os.environ` was never called. The env var was set but completely ignored by the code.

The two-step fix:

1. Update the source code to read from env vars (already done in the repo by Kiro's edit during this session)
2. Re-zip and re-deploy the Lambda functions:

```
cd lambda-functions/generate-upload-url
zip ../../build/generate_upload_url.zip lambda_function.py && cd ../..
aws lambda update-function-code \
  --function-name generate-upload-url \
  --zip-file fileb://build/generate_upload_url.zip \
  --region ap-south-1

cd lambda-functions/get-processed-images
zip ../../build/get_processed_images.zip lambda_function.py && cd ../..
aws lambda update-function-code \
  --function-name get-processed-images \
  --zip-file fileb://build/get_processed_images.zip \
  --region ap-south-1
```

Verify the fix worked:

Run the curl test again — the uploadUrl should now say `image-pipeline-input-`

`anshuman.s3.amazonaws.com`.

Lesson: setting env vars on a Lambda that doesn't read them does nothing.

Always check whether the function code actually calls `os.environ.get("VAR_NAME")`. If it has hardcoded values, the code must be updated and redeployed.

Issue Hit: Error parsing parameter '--environment' — wrong quoting for update-function-configuration

```
# These both failed:
--environment Variables={"INPUT_BUCKET":"image-pipeline-input-anshuman"}
--environment 'Variables={"INPUT_BUCKET":"image-pipeline-input-anshuman"}'

# Error:
Error parsing parameter '--environment': Expected: '=', received: '"'
```

Why:

The `--environment` flag on `update-function-configuration` expects the entire value as a JSON object wrapped in `{"Variables":{...}}` format — not the shorthand `Variables={...}` format. The CLI's shorthand parser gets confused by the `{` after the `=`.

Correct format — wrap in {"Variables":{...}} with single quotes:

```
aws lambda update-function-configuration \
  --function-name generate-upload-url \
  --region ap-south-1 \
  --environment '{"Variables":{"INPUT_BUCKET":"image-pipeline-input-anshuman"}}'
```

Note: single quotes work here because the JSON contains no shell variables — the bucket name is a literal string, not a `$VARIABLE`.

2 Test the actual upload — PUT a file directly to S3 using the pre-signed URL

```
# Step 2a: Get a pre-signed URL (save the uploadUrl value)
UPLOAD_URL=$(curl -s -X POST $API_BASE/generate-upload-url \
  -H "Content-Type: application/json" \
  -d '{"filename": "test-image.jpg", "contentType": "image/jpeg"}' \
  | python3 -c "import sys,json; print(json.load(sys.stdin)['uploadUrl'])")

# Step 2b: Upload a real image file directly to S3 using that URL
curl -X PUT "$UPLOAD_URL" \
  -H "Content-Type: image/jpeg" \
  --data-binary @/path/to/any/test-image.jpg
```

WHY TEST THE UPLOAD SEPARATELY FROM THE UI?

This verifies two things: (1) the pre-signed URL is valid and S3 accepts it, and (2) CORS is configured correctly on the input bucket. If curl succeeds but the browser fails, it's a CORS issue on the bucket. If curl also fails with 403, the pre-signed URL is malformed or the IAM role

lacks s3:PutObject permission.

After a successful upload, check the input bucket: `aws s3 ls s3://$INPUT_BUCKET/ --region $REGION` — you should see test-image.jpg listed.

3 Verify the S3 event triggered ImageProcessor automatically

```
# Wait 10-15 seconds after upload, then check if processed files appeared
aws s3 ls s3://$OUTPUT_BUCKET/1080p/ --region $REGION
aws s3 ls s3://$OUTPUT_BUCKET/720p/ --region $REGION
aws s3 ls s3://$OUTPUT_BUCKET/480p/ --region $REGION

# Check CloudWatch logs for the ImageProcessor
aws logs tail /aws/lambda/ImageProcessorLambda-Shivam \
  --region $REGION \
  --since 5m
```

WHAT YOU'RE CONFIRMING HERE

If processed files appear in all three folders: the S3 event trigger works, the ImageProcessor Lambda was invoked automatically, IAM permissions allow reading from input and writing to output, and Pillow processed the image correctly. If files DON'T appear: check CloudWatch logs for errors. Common causes: Pillow not bundled correctly in the ZIP (ImportError), IAM role lacks s3:GetObject on input or s3:PutObject on output, or the PNG transparency bug (RGBA mode).

4 Test Lambda 3 — get-processed-images

```
curl $API_BASE/processed-images/test-image.jpg | python3 -m json.tool
```

EXPECTED RESPONSE

Expected: `{"1080p": "https://image-pipeline-output-$SUFFIX.s3.amazonaws.com/1080p/test-image.jpg?X-Amz-...", "720p": "...", "480p": "..."}`. Click each URL in a browser to confirm the processed images download correctly. If you get 404 from Lambda — the processed files don't exist yet (ImageProcessor hasn't finished or failed). If you get 500 — check CloudWatch logs for Lambda 3.

5 Test the frontend end-to-end

```
# Open in browser:
echo "http://$FRONTEND_BUCKET.s3-website.$REGION.amazonaws.com"
```

WHAT TO VERIFY IN THE BROWSER

Upload a test image (JPEG first, then try PNG). Wait ~5 seconds. Click the three download buttons. Compare the original and processed file sizes — you should see 90%+ reduction. If the upload button does nothing: open browser DevTools → Network tab — look for failed requests. Common issues: API URL still has the old hardcoded value (ijlmwfo9gd), CORS not configured on the input bucket, or the S3 bucket policy not allowing public access.

11 Data Flow & Key Configs Reference

Config Key	Value	Used By
Region	ap-south-1	Every AWS CLI command
Input bucket name	image-pipeline-input-$SUFFIX	Lambda 1 (generates pre-signed PUT URL), S3 event trigger
Output bucket name	image-pipeline-output-$SUFFIX	ImageProcessor (uploads to), Lambda 3 (generates pre-signed GET URLs from)
Frontend bucket	image-pipeline-frontend-$SUFFIX	S3 static website hosting
Lambda 1 name	generate-upload-url	API Gateway POST /generate-upload-url
Lambda 2 name	ImageProcessorLambda-Shivam	S3 event trigger only
Lambda 3 name	get-processed-images	API Gateway GET /processed-images/{filename}
IAM Role name	Lambda-Image-Processing-Role	All 3 Lambda functions
Output subfolders	1080p/ 720p/ 480p/	ImageProcessor writes here; Lambda 3 reads from here
Compression quality	85 (out of 100)	ImageProcessor env var COMPRESSION_QUALITY
Pre-signed URL expiry (upload)	900 seconds (15 min)	Lambda 1 — user must upload within 15 minutes
Pre-signed URL expiry (download)	3600 seconds (1 hour)	Lambda 3 — download links valid for 1 hour
API Gateway stage	prod	API URL path: /prod/generate-upload-url

PNG Transparency Bug — What It Is and How It's Fixed

Bug: "cannot write mode RGBA as JPEG" — affects all PNG files

PNG images can be saved in RGBA mode (Red, Green, Blue, Alpha). JPEG format does not support an alpha channel. When the ImageProcessor tried to save an RGBA image as JPEG, Pillow threw a fatal error and the entire processing failed — no output files were created.

Fix already in lambda_function.py (lines 29-36):

```
# After resize, before saving — convert mode if needed
if resized_image.mode == 'RGBA':
    rgb_image = Image.new('RGB', resized_image.size, (255, 255, 255))  # white bg
    rgb_image.paste(resized_image, mask=resized_image.split()[3])      # alpha as mask
    resized_image = rgb_image
elif resized_image.mode != 'RGB':
    resized_image = resized_image.convert('RGB')  # grayscale, CMYK, etc.
```

Transparent areas become white. All image formats (JPEG, PNG, BMP, TIFF, GIF, WebP) now process successfully. This fix is already in the repo — you do not need to add it. But you MUST know about it for an interview.

12 Troubleshooting Quick Fixes

Problem	Root Cause	Fix
S3 bucket creation fails with BucketAlreadyExists	Name taken globally	Add a unique suffix to the bucket name (date, username). Update all references to the new name in Lambda env vars and the notification configuration.
create-bucket fails with LocationConstraint error	Missing LocationConstraint for non-us-east-1	Add `--create-bucket-configuration LocationConstraint=ap-south-1`. This is required for every region except us-east-1.
Upload button fails in browser with CORS error	Input bucket missing CORS configuration	Re-run Phase 1 Step 4 (put-bucket-cors on input bucket). Check browser DevTools Network tab for the preflight OPTIONS response.
Frontend returns 403 Forbidden	Public access still blocked on frontend bucket	Re-run Phase 1 Steps 3a and 3b — both the block public access setting AND the bucket policy must be applied.
Lambda 2 ImportError: cannot import _imaging from PIL	Pillow bundled for wrong OS (Mac binary in Linux Lambda)	Delete the image-processor-pkg directory, re-run pip install WITH the `--platform manylinux2014_x86_64` flag, re-zip, re-deploy.
S3 event does not trigger ImageProcessor	Lambda permission not added before S3 notification	Re-run Phase 4 Step 1 (lambda add-permission), then re-run Phase 4 Step 2 (put-bucket-notification-configuration).
API returns {"message": "Missing	API Gateway not deployed to stage yet	Run `aws apigateway create-deployment --rest-api-id $API_ID --stage-name prod --region $REGION`

Authentication Token"}		
API changes not taking effect	REST API v1 requires explicit re-deployment after any change	Every change to methods, integrations, or CORS requires a new deployment: `aws apigateway create-deployment --rest-api-id $API_ID --stage-name prod --region $REGION`
500 from API — Lambda not invoked	Lambda invoke permission not granted to API Gateway	Re-run the `aws lambda add-permission` commands from Phase 5. Verify with `aws lambda get-policy --function-name generate-upload-url --region $REGION`
Processed files not appearing after upload	ImageProcessor error — check CloudWatch	`aws logs tail /aws/lambda/ImageProcessorLambda-Shivam --region $REGION --since 10m` — look for the specific error (usually Pillow import, RGBA mode, or IAM permission)
cannot write mode RGBA as JPEG	PNG transparency bug — code not updated	Verify lambda_function.py in image-processor includes the RGBA conversion block. Re-package and re-deploy Lambda 2.

🔍 CloudWatch is your first stop for any Lambda failure:

```
aws logs tail /aws/lambda/generate-upload-url --region $REGION --follow
aws logs tail /aws/lambda/ImageProcessorLambda-Shivam --region $REGION --follow
aws logs tail /aws/lambda/get-processed-images --region $REGION --follow
```

The `--follow` flag streams live logs. Press Ctrl+C to stop.

13 Interview Master Checklist

- [ ] SETUP AWS CLI works — `aws sts get-caller-identity` returns account ID
- [ ] SETUP Shell variables set: `REGION=ap-south-1` and `ACCOUNT_ID`
- [ ] S3 3 buckets created: input, output, frontend
- [ ] S3 Frontend bucket has website hosting enabled and public read policy
- [ ] S3 Input bucket has CORS for PUT operations (browser direct upload)
- [ ] S3 Output bucket has CORS for GET operations (browser direct download)
- [ ] IAM Lambda-Image-Processing-Role created with Lambda trust policy
- [ ] IAM AWSLambdaBasicExecutionRole attached (CloudWatch Logs)
- [ ] IAM S3 read/write inline policy attached
- [ ] IAM `ROLE_ARN` variable saved
- [ ] LAMBDA Lambda 1 (generate-upload-url) deployed — simple ZIP, no dependencies

- [ ] LAMBDA Lambda 3 (get-processed-images) deployed — simple ZIP, no dependencies
- [ ] LAMBDA Lambda 2 (ImageProcessor) packaged with Pillow Linux binary using manylinux2014_x86_64 platform flag
- [ ] LAMBDA Lambda 2 deployed with 1024 MB memory and 59 second timeout
- [ ] TRIGGER Lambda invoke permission granted to s3.amazonaws.com
- [ ] TRIGGER S3 bucket notification configured on input bucket → ImageProcessorLambda
- [ ] API REST API (v1) created — noted `API_ID` and `ROOT_ID`
- [ ] API /generate-upload-url resource + POST method + Lambda 1 integration
- [ ] API /generate-upload-url OPTIONS method with MOCK CORS integration
- [ ] API /processed-images/{filename} resource + GET method + Lambda 3 integration
- [ ] API /processed-images/{filename} OPTIONS method with MOCK CORS integration
- [ ] API Lambda invoke permissions granted to API Gateway for Lambdas 1 and 3
- [ ] API API deployed to "prod" stage — `create-deployment` run
- [ ] FRONTEND API_BASE_URL in app.js updated to real API Gateway URL
- [ ] FRONTEND Files synced to frontend bucket with correct content types
- [ ] TEST POST /generate-upload-url returns a pre-signed URL
- [ ] TEST curl PUT to pre-signed URL uploads file to S3
- [ ] TEST Processed files appear in output bucket within 10 seconds
- [ ] TEST GET /processed-images/filename returns 3 download URLs
- [ ] TEST Frontend loads, uploads a real image, downloads work — both JPEG and PNG

14 Talking Points for Interviewers

Why pre-signed URLs instead of uploading through API Gateway?

API Gateway has a 10 MB request body limit. Large images would fail. Pre-signed URLs let the browser upload directly to S3 — no size limit, no API Gateway bottleneck, no bandwidth cost through API Gateway. The API just generates the credential, not the data.

Why S3 event trigger instead of the API calling Lambda?

Decoupling. The upload and processing are independent operations. Processing is asynchronous — the user doesn't wait. The same pipeline works regardless of how the image arrives (UI, CLI, another system). Serverless event-driven design: each service does one thing and reacts to events.

Why Pillow at quality 85 and not 100?

JPEG quality is perceptually logarithmic — 85 is visually near-identical to 100 but produces 70–90% smaller files. Quality 100 produces lossless-like files with minimal compression. 85 is the industry standard for web images — Netflix, YouTube, and most CDNs use similar settings.

Why 3 separate S3 buckets?

Separation of concerns + security. Input bucket fires S3 events — it would create an infinite loop if the ImageProcessor wrote back to it. Output bucket needs different CORS (GET only). Frontend needs public read. Mixing them violates least-privilege and creates debugging nightmares.

Why REST API v1 and not HTTP API v2?

REST API v1 was the original choice — it existed before v2 and offers more features (request validation, API keys, usage plans, resource policies). The project predates the v2 adoption. In a greenfield project today, HTTP API v2 would be the better choice for a simple proxy pattern like this — cheaper, simpler CORS, automatic deployment.

How does the PNG transparency fix work?

PNG supports RGBA (with alpha channel). JPEG does not support transparency. PIL raises an error if you try to save RGBA as JPEG. The fix: create a white RGB background canvas, paste the RGBA image using the alpha channel as a mask (transparent pixels become white), then save as JPEG. All formats now supported.

Why 1024 MB memory for ImageProcessor?

Two reasons: (1) A 14 MB JPEG decompressed into raw pixels = ~100-200 MB in RAM. Processing 3 resolutions risks OOM at 128 MB. (2) Lambda CPU is proportional to memory — more memory = more CPU = faster Pillow operations. 1024 MB struck the right balance of speed vs cost for the testing results we saw.

What would you add next?

WebP output (30% smaller than JPEG at same quality). CloudFront CDN in front of both S3 buckets for HTTPS and edge caching. SNS/SQS for async notification when processing completes (instead of the frontend polling and waiting). Custom resolution selection. Batch processing via S3 batch operations.

🎯 **Narrate every step in an interview:**

"I'm creating the input bucket first because the IAM role needs to reference its ARN in the permissions policy. Then I'll create the IAM role so Lambda can be created with permissions. Notice the image never passes through API Gateway — it goes directly from the browser to S3 via the pre-signed URL, which removes the 10 MB Gateway limit entirely." Architecture reasoning out loud = confidence.

⏱ **Time estimate with all tools pre-installed:**

S3 setup 3 min → IAM 2 min → Lambda packaging + deploy 5 min (Pillow download is slow) → S3 trigger 1 min → API Gateway 5 min → Frontend 2 min → Testing 3 min →
~21 minutes total.

The Pillow pip install is the bottleneck — it downloads ~5MB of Linux binaries.